

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа №2
«Реализация микросервисов»

Выполнил:

Якшин Артемий

Группа БР1.1

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Опираясь на технический дизайн микросервисной архитектуры из ДЗ4, реализовать разделение монолитного бэкенд-приложения «Restaurant Booking» (бронирование столиков, ЛР1) на микросервисы по принципу *database-per-service*: каждый сервис — отдельное приложение со своей базой данных; межсервисное взаимодействие — по спроектированному внутреннему контракту; единая точка входа для клиента — API Gateway. Внешний публичный контракт (ЛР1/ДЗ2) при этом не меняется.

Ход работы

1. Состав системы

Реализовано 5 сервисов (Node.js + TypeScript + Express + TypeORM + SQLite), каждый — самостоятельное приложение:

Сервис	Порт	Своя БД (SQLite)	Ответственность
gateway (API Gateway)	3000	—	единая точка входа, маршрутизация <code>/api/v1/*</code> на сервисы
auth-service	8081	auth.sqlite	регистрация/вход, выпуск JWT, профиль пользователя
catalog-service	8082	catalog.sqlite	рестораны, кухни, меню, фото, столики; поиск/фильтрация
reservation-service	8083	reservations.sqlite	создание/изменение/отмена бронирований, проверка занятости
review-service	8084	reviews.sqlite	отзывы, расчёт среднего рейтинга

2. Структура репозитория

```
labs/lab2/
├── docker-compose.yml      # оркестрация всех 5 сервисов (ЛР3)
├── .env.example            # JWT_SECRET, INTERNAL_API_KEY, ...
├── README.md
├── gateway/                # Dockerfile + src/index.ts (http-proxy-middleware)
├── auth-service/
│   ├── Dockerfile, package.json, tsconfig.json
│   └── src/{index.ts, data-source.ts, common.ts, entities/User.ts}
├── catalog-service/
│   └── src/{index.ts, data-source.ts, common.ts,
│       entities/{Restaurant, Cuisine, MenuItem, RestaurantPhoto, RestaurantTable}.ts}
├── reservation-service/
│   └── src/{index.ts, data-source.ts, common.ts, entities/Reservation.ts}
└── review-service/
    └── src/{index.ts, data-source.ts, common.ts, entities/Review.ts}
```

Каждый сервис содержит свой `data-source.ts` (отдельный `DataSource` TypeORM на свой SQLite-файл, `synchronize: true`), набор сущностей только своего контекста, модуль `common.ts` (типизированные ошибки, обработчик ошибок, проверка JWT, middleware сервисного ключа, межсервисный HTTP-клиент с таймаутом). Данные не пересекаются: ни одной общей таблицы между сервисами нет.

3. Модели и разделение БД (database-per-service)

Сервис / БД	Сущности (таблицы)	Внешние ссылки
auth-service / <code>auth.sqlite</code>	<code>users</code> (<code>user_id</code> , name, email уник., phone, password_hash, created_at)	—
catalog-service / <code>catalog.sqlite</code>	<code>restaurants</code> , <code>cuisines</code> , <code>restaurant_cuisines</code> (M:N), <code>menu_items</code> , <code>restaurant_photos</code> , <code>restaurant_tables</code>	FK только внутри контекста — остаются обычными FK
reservation-service / <code>reservations.sqlite</code>	<code>reservations</code> (<code>reservation_id</code> , <u><code>user_id</code></u> , <u><code>table_id</code></u> , <u><code>restaurant_id</code></u> , date, time, guests_count, status, created_at)	<u><code>user_id</code></u> → auth, <u><code>table_id/restaurant_id</code></u> → catalog: числовые ссылки без FK
review-service / <code>reviews.sqlite</code>	<code>reviews</code> (<code>review_id</code> , <u><code>user_id</code></u> , <u><code>restaurant_id</code></u> , rating, comment, created_at; уникальность пары <code>user_id+restaurant_id</code>)	<u><code>user_id</code></u> → auth, <u><code>restaurant_id</code></u> → catalog: числовые ссылки без FK

Целостность «осиротевших» ссылок обеспечивается проверками через внутренний API при записи (см. §4): `reservation` проверяет существование столика у `catalog`, `review` — существование ресторана; обогащение ответов (рейтинги, имена авторов, карточка ресторана) делается тоже через внутренний API. На старте каждый сервис при пустой БД наполняет её своими тестовыми данными (auth — 2 пользователя, catalog — 4 ресторана с меню/фото/столиками, review — 3 отзыва).

4. Межсервисное взаимодействие

Реализовано **синхронное** взаимодействие по REST/HTTP-JSON. Внутренние эндпоинты вынесены на путь `/internal/*`, наружу через gateway не выставляются, защищены заголовком `X-Internal-Key` (общий секрет `INTERNAL_API_KEY`). Адреса соседей передаются через переменные окружения. Клиентский JWT подписывается `auth-service` и проверяется `reservation-service/review-service` по общему `JWT_SECRET`.

Вызывающий	Вызываемый эндпоинт	Когда / зачем
reservation → catalog	GET <code>/internal/tables/:id</code>	при создании/изменении брони — проверить существование столика,

		узнать <code>capacity</code> и <code>restaurant_id</code>
reservation → catalog	GET <code>/internal/restaurants/:id</code>	обогащение ответа бронирования карточкой ресторана
catalog → reservation	POST <code>/internal/reservations/availability-batch</code>	в GET <code>/restaurants/:id/tables</code> — пометить столики <code>available/reserved</code>
catalog → review	POST <code>/internal/ratings/batch</code>	в списке и карточке ресторанов — добавить <code>average_rating</code> , <code>reviews_count</code>
review → catalog	GET <code>/internal/restaurants/:id</code>	при создании отзыва — проверить существование ресторана
review → auth	POST <code>/internal/users/batch</code>	обогащение списка отзывов именами авторов

Деградация. Межсервисный клиент имеет таймаут. Для *обогащения* ответа недоступность соседа не критична (рейтинг → 0, имя автора → только id, столики → считаются свободными). Для операций, где проверка обязательна (создание брони/отзыва), недоступность соседа возвращается клиенту как 503 Service Unavailable, отсутствие ресурса — как 404.

5. API Gateway

Реализован на `express` + `http-proxy-middleware`. Тело запроса проксируется потоком (парсер тела в gateway намеренно не подключён). Маршрутизация по префиксу пути с учётом перекрытий: `/api/v1/users/me/reservations` → reservation (раньше, чем `/users` → auth), `/api/v1/restaurants/:id/reviews` → review (раньше, чем `/restaurants` → catalog). Также gateway отдаёт GET `/` с картой маршрутов и GET `/health`. Сервисы монтируют публичные маршруты под тем же префиксом `/api/v1`, поэтому переписывание пути не требуется.

```
function pick(path) {
  if (path.startsWith('/api/v1/auth')) return 'auth';
  if (path.startsWith('/api/v1/users/me/reservations')) return 'reservation';
  if (path.startsWith('/api/v1/users')) return 'auth';
  if (/^\/api\/v1\/restaurants\/\d+\/reviews(\||$)/.test(path)) return 'review';
  if (path.startsWith('/api/v1/restaurants') || path.startsWith('/api/v1/cuisines'))
    return 'catalog';
  if (path.startsWith('/api/v1/reservations')) return 'reservation';
  if (path.startsWith('/api/v1/reviews')) return 'review';
}
```

6. Отличия от монолита

- единая БД → 4 независимых БД; межконтекстные ФК заменены на числовые ссылки + проверка через API;

- прямые JOIN/обращения к чужим таблицам → межсервисные REST-вызовы (с таймаутами и мягкой деградацией);
- добавлен слой API Gateway; добавлена сервисная аутентификация (`X-Internal-Key`);
- проверка JWT вынесена в каждый сервис, который этого требует (общий секрет);
- сидинг разнесён по сервисам (каждый наполняет только свою БД).

7. Запуск

```
cd "BP1.1/Якшин Артемий/labs/lab2"
docker compose up --build          # все 5 сервисов; gateway стартует после healthy остальных
# API:      http://localhost:3000/api/v1
# карта маршрутов: GET http://localhost:3000/
# health: GET http://localhost:3000/health (а также :8081...:8084/health внутри сети)
```

(Без Docker — в каждом каталоге `npm i && npm run dev`; порты по умолчанию 8081–8084 и 3000 уже разведены.)

8. Проверка работоспособности

Стек поднят через `docker compose`, сквозной сценарий выполнен через gateway (порт 3000):

```

$ docker compose ps
NAME                                SERVICE                STATUS
lab2-auth-service-1                auth-service            Up (healthy)
lab2-catalog-service-1             catalog-service          Up (healthy)
lab2-reservation-service-1         reservation-service     Up (healthy)
lab2-review-service-1              review-service          Up (healthy)
lab2-gateway-1                     gateway                 Up    0.0.0.0:3000->3000/tcp

$ curl -s http://localhost:3000/                                # карта маршрутов gateway
{"service":"api-gateway","routes":[{"POST /api/v1/auth/...":"http://auth-service:8081",
... }]}

$ TOKEN=$(curl -s -X POST .../auth/login -d
'{"email":"ivan@example.com","password":"securepass123"}' | jq -r .token)
$ curl -s .../users/me -H "Authorization: Bearer $TOKEN"        # [auth-service]
{"user_id":1,"name":"Иван Иванов","email":"ivan@example.com","phone":"+79001234567",
...}

$ curl -s --get --data-urlencode "city=Москва" .../restaurants # catalog + рейтинги из
review
total 3
1 La Piazza    avg= 4.5  photo= https://example.com/photos/lapiazza-1.jpg
3 Тбилисо     avg= 5    photo= https://example.com/photos/tbiliso-1.jpg
4 Самовар     avg= 0    photo= https://example.com/photos/samovar-1.jpg

$ curl -s .../restaurants/1                                     # catalog detail
name La Piazza  avg 4.5  reviews_count 2  menu_items 3  photos 2

$ curl -s ".../restaurants/1/tables?date=2026-07-01&time=19:00" # catalog +
availability из reservation
[{"table_id":1,"capacity":2,"status":"available"}, ... 5 столиков ...]

$ curl -s -X POST .../reservations -H "Authorization: Bearer $TOKEN" \
-d '{"table_id":1,"reservation_date":"2026-07-
01","reservation_time":"19:00","guests_count":2}'
{"reservation_id":1,"user_id":1,"table":{"table_id":1},
"restaurant":{"restaurant_id":1,"name":"La Piazza","city":"Москва", ...}, # ←
обращено из catalog
"reservation_date":"2026-07-01","reservation_time":"19:00","status":"confirmed", ...}

$ curl -s ".../restaurants/1/tables?date=2026-07-01&time=19:00" # тот же запрос —
стол 1 уже занят
[{"table_id":1,"capacity":2,"status":"reserved"},
{"table_id":2,...,"status":"available"}, ...]

$ curl -s -X POST .../reservations ... (тот же слот)           # конфликт
HTTP 409 {"error":"Conflict","message":"Table is already booked for this date and
time"}

$ curl -s .../users/me/reservations -H "Authorization: Bearer $TOKEN" # [reservation-
service]
{"data":[{"reservation_id":1, "restaurant":{"name":"La Piazza",...}, ...],"total":1,
...}

$ curl -s -X POST .../reviews -H "Authorization: Bearer $TOKEN" \
-d '{"restaurant_id":2,"rating":5,"comment":"Отличный японский ресторан"}' #
review → catalog (валидация) → auth (автор)
{"review_id":4,"user":{"user_id":1,"name":"Иван Иванов",
...},"restaurant_id":2,"rating":5, ...}

$ curl -s .../restaurants/1/reviews                             # review + имена
авторов из auth
total 2  avg 4.5
1 Иван Иванов    5  "Отличный ресторан, вкусная еда!"

```

```
2 Мария Петрова 4 "Хорошее место, но чуть шумно."
```

```
$ curl -s .../restaurants/999 → HTTP 404 {"error":"Not Found", ...}  
$ curl -s .../users/me → HTTP 401 {"error":"Unauthorized", ...}
```

Все межсервисные взаимодействия отработали: gateway корректно маршрутизирует на 4 сервиса; в карточке и списке ресторанов появились рейтинги (из `review`) и главное фото; в выдаче столиков — актуальная занятость (из `reservation`); бронирование валидирует столик и обогащается рестораном (из `catalog`); отзыв валидирует ресторан и обогащается автором (из `auth`); коды ошибок (404/401/409/503) корректны. Сборка образов (`tsc`) проходит без ошибок.

Вывод

Опираясь на технический дизайн из ДЗ4, монолит «Restaurant Booking» разделён на пять независимых сервисов (API Gateway + Auth, Catalog, Reservation, Review), каждый — со своей базой данных по принципу database-per-service. Реализовано синхронное межсервисное взаимодействие по REST с сервисной аутентификацией и мягкой деградацией; межконтекстные внешние ключи заменены на числовые ссылки, проверяемые через внутренний API. API Gateway обеспечивает единую точку входа без изменения публичного контракта. Работоспособность системы подтверждена сквозным сценарием через gateway (регистрация/вход → каталог с рейтингами → выбор столика → бронирование → история → отзыв), все кросс-сервисные вызовы и обработка ошибок работают корректно. Проект подготовлен к контейнеризации (ЛР3).